



Guru and Student...

Guru: Have we talked about loose coupling?

Student: Guru, I do not recall such a discussion.

Guru: Is a tightly woven basket stiff or flexible?

Student: Stiff, Guru.

Guru: And do stiff or flexible baskets tear or break less easily?

Student: A flexible basket tends to break less easily.

Guru: And in our software, might our designs break less easily if our objects are less tightly bound together?

Student: Guru, I see the truth of it. But what does it mean for objects to be less tightly bound?

Guru: We like to call it, loosely coupled.

Student: Ah!

Guru: We say a object is tightly coupled to another object when it is **too** dependent on that object.

Student: So a loosely coupled object can't depend on another object?

Guru: Think of nature; all living things depend on each other. Likewise, all objects depend on other objects. But a loosely coupled object doesn't know or care too much about the details of another object.

Student: But Guru, that doesn't sound like a good quality. Surely not knowing is worse than knowing.

Guru: You are doing well in your studies, but you have much to learn. By not knowing too much about other objects, we can create designs that can handle change better. Designs that have more flexibility, like the less tightly woven basket.

Student: Of course, I am sure you are right. Could you give me an example?

Guru: That is enough for today.

The Power of Loose Coupling

When two objects are *loosely coupled*, they can interact, but they typically have very little knowledge of each other. As we're going to see, loosely coupled designs often give us a lot of flexibility (more on that in a bit). And, as it turns out, the Observer Pattern is a great example of loose coupling. Let's walk through all the ways the pattern achieves loose coupling:

First, the only thing the subject knows about an observer is that it implements a certain interface (the Observer interface). It doesn't need to know the concrete class of the observer, what it does, or anything else about it.

We can add new observers at any time. Because the only thing the subject depends on is a list of objects that implement the Observer interface, we can add new observers whenever we want. In fact, we can replace any observer at runtime with another observer and the subject will keep purring along. Likewise, we can remove observers at any time.

We never need to modify the subject to add new types of observers. Let's say we have a new concrete class come along that needs to be an observer. We don't need to make any changes to the subject to accommodate the new class type; all we have to do is implement the Observer interface in the new class and register as an observer. The subject doesn't care; it will deliver notifications to any object that implements the Observer interface.

How many different kinds of change can you identify here?

We can reuse subjects or observers independently of each other. If we have another use for a subject or an observer, we can easily reuse them because the two aren't tightly coupled.

Changes to either the subject or an observer will not affect the other.

Because the two are loosely coupled, we are free to make changes to either, as long as the objects still meet their obligations to implement the Subject or Observer interfaces.



Look! We have a new Design Principle!

Loosely coupled designs allow us to build flexible OO systems that can handle change because they minimize the interdependency between objects.

Sharpen your pencil

Before moving on, try sketching out the classes you'll need to implement the Weather Station, including the WeatherData class and its display elements. Make sure your diagram shows how all the pieces fit together and also how another developer might implement her own display element.

If you need a little help, read the next page; your teammates are already talking about how to design the Weather Station.

Cubicle conversation

Back to the Weather Station project. Your teammates have already begun thinking through the problem...



Mary: Well, it helps to know we're using the Observer Pattern.

Sue: Right...but how do we apply it?

Mary: Hmm. Let's look at the definition again:

The Observer Pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Mary: That actually makes some sense when you think about it. Our WeatherData class is the "one," and our "many" is the various display elements that use the weather measurements.

Sue: That's right. The WeatherData class certainly has state...that's the temperature, humidity, and barometric pressure, and those definitely change.

Mary: Yup, and when those measurements change, we have to notify all the display elements so they can do whatever it is they are going to do with the measurements.

Sue: Cool, now I think I see how the Observer Pattern can be applied to our Weather Station problem.

Mary: There are still a few things to consider that I'm not sure I understand yet.

Sue: Like what?

Mary: For one thing, how do we get the weather measurements to the display elements?

Sue: Well, looking back at the picture of the Observer Pattern, if we make the WeatherData object the subject, and the display elements the observers, then the displays will register themselves with the WeatherData object in order to get the information they want, right?

Mary: Yes...and once the Weather Station knows about a display element, then it can just call a method to tell it about the measurements.

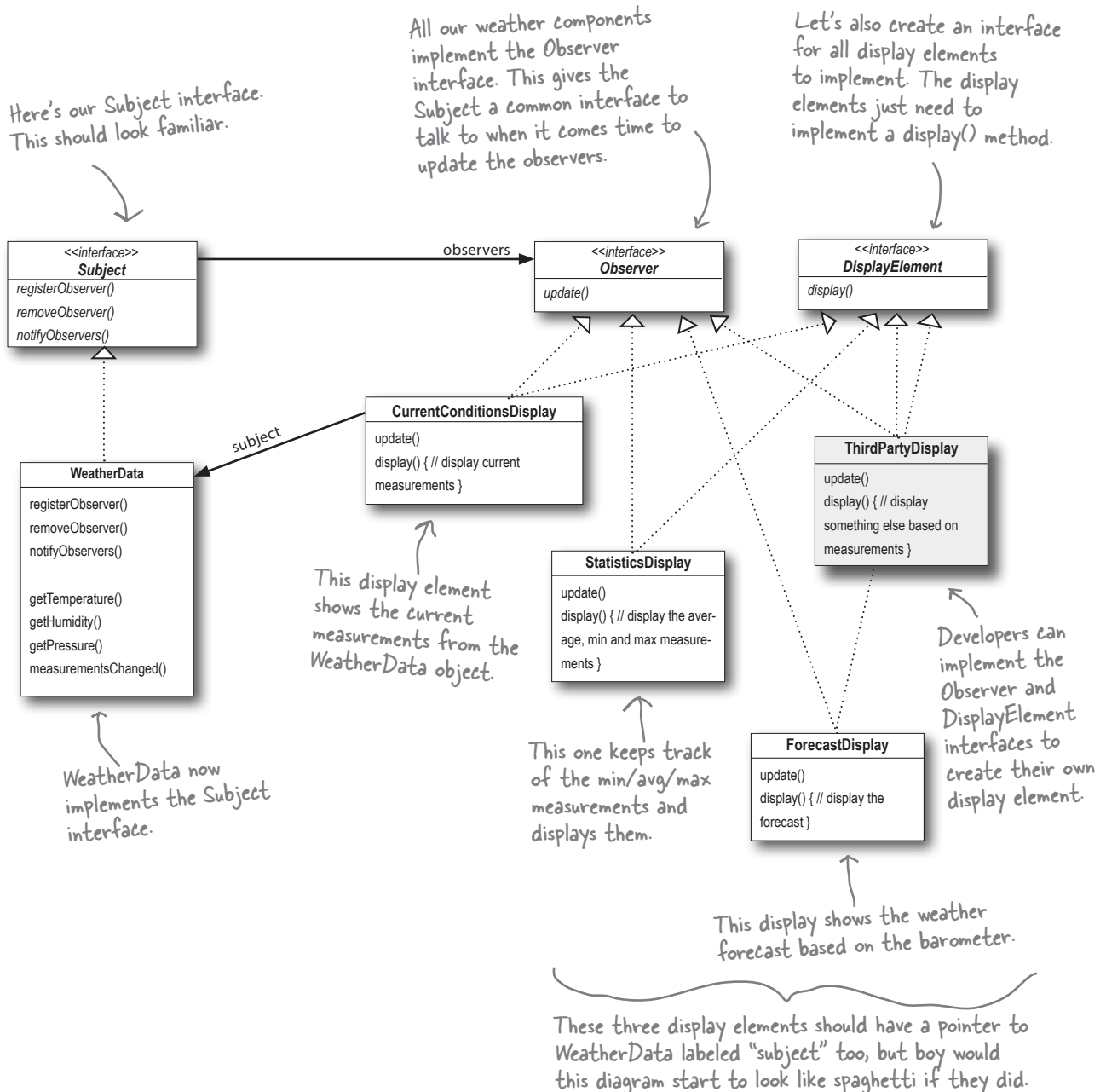
Sue: We gotta remember that every display element can be different...so I think that's where having a common interface comes in. Even though every component has a different type, they should all implement the same interface so that the WeatherData object will know how to send them the measurements.

Mary: I see what you mean. So every display will have, say, an `update()` method that WeatherData will call.

Sue: And `update()` is defined in a common interface that all the elements implement...

Designing the Weather Station

How does this diagram compare with yours?



Implementing the Weather Station

All right, we've had some great thinking from Mary and Sue (from a few pages back) and we've got a diagram that details the overall structure of our classes. So, let's get our implementation of the weather station underway. Let's start with the interfaces:

```
public interface Subject {
    public void registerObserver(Observer o);
    public void removeObserver(Observer o);
    public void notifyObservers();
}
```

Both of these methods take an *Observer* as an argument—that is, the *Observer* to be registered or removed.

This method is called to notify all observers when the *Subject's* state has changed.

```
public interface Observer {
    public void update(float temp, float humidity, float pressure);
}
```

These are the state values the *Observers* get from the *Subject* when a weather measurement changes.

The *Observer* interface is implemented by all observers, so they all have to implement the *update()* method. Here we're following Mary and Sue's lead and passing the measurements to the observers.

```
public interface DisplayElement {
    public void display();
}
```

The *DisplayElement* interface just includes one method, *display()*, that we will call when the display element needs to be displayed.



Mary and Sue thought that passing the measurements directly to the observers was the most straightforward method of updating state. Do you think this is wise? Hint: is this an area of the application that might change in the future? If it did change, would the change be well encapsulated, or would it require changes in many parts of the code?

Can you think of other ways to approach the problem of passing the updated state to the observers?

Don't worry; we'll come back to this design decision after we finish the initial implementation.

Implementing the Subject interface in WeatherData

Remember our first attempt at implementing the WeatherData class at the beginning of the chapter? You might want to refresh your memory. Now it's time to go back and do things with the Observer Pattern in mind:

REMEMBER: we don't provide import and package statements in the code listings. Get the complete source code from <https://wickedlysmart.com/head-first-design-patterns>

```
public class WeatherData implements Subject {
    private List<Observer> observers;
    private float temperature;
    private float humidity;
    private float pressure;

    public WeatherData() {
        observers = new ArrayList<Observer>();
    }

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(temperature, humidity, pressure);
        }
    }

    public void measurementsChanged() {
        notifyObservers();
    }

    public void setMeasurements(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        measurementsChanged();
    }

    // other WeatherData methods here
}
```

Here we implement the Subject interface.

WeatherData now implements the Subject interface.

We've added an ArrayList to hold the Observers, and we create it in the constructor.

When an observer registers, we just add it to the end of the list.

Likewise, when an observer wants to un-register, we just take it off the list.

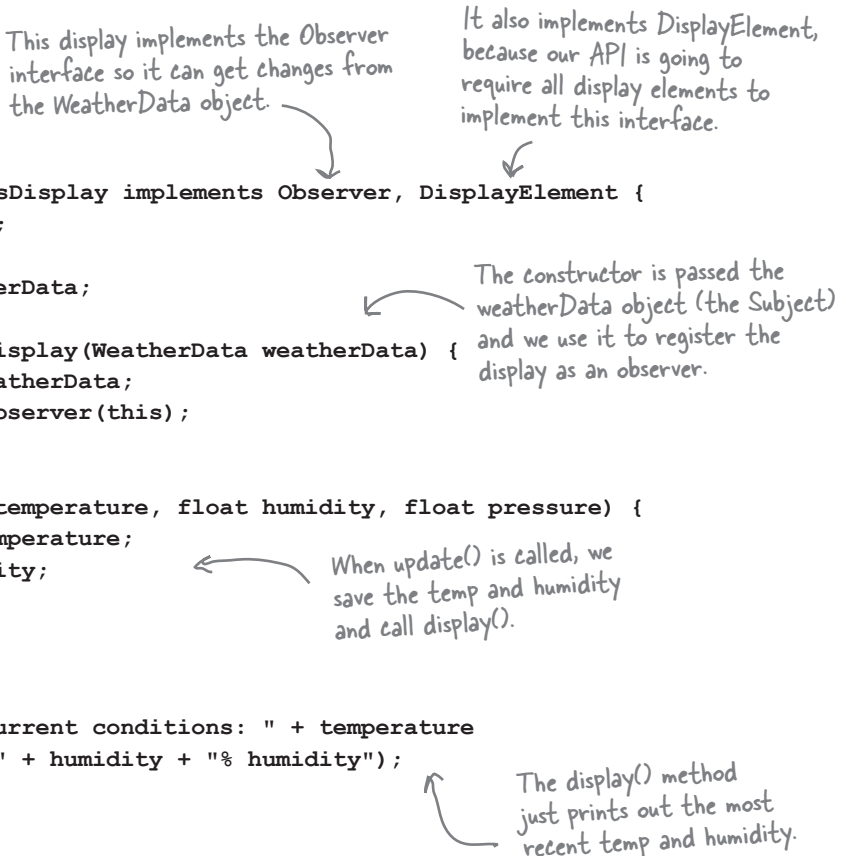
Here's the fun part; this is where we tell all the observers about the state. Because they are all Observers, we know they all implement update(), so we know how to notify them.

We notify the Observers when we get updated measurements from the Weather Station.

Okay, while we wanted to ship a nice little weather station with each book, the publisher wouldn't go for it. So, rather than reading actual weather data off a device, we're going to use this method to test our display elements. Or, for fun, you could write code to grab measurements off the web.

Now, let's build those display elements

Now that we've got our `WeatherData` class straightened out, it's time to build the display elements. Weather-O-Rama ordered three: the current conditions display, the statistics display, and the forecast display. Let's take a look at the current conditions display; once you have a good feel for this display element, check out the statistics and forecast displays in the code directory. You'll see they are very similar.



```
public class CurrentConditionsDisplay implements Observer, DisplayElement {
    private float temperature;
    private float humidity;
    private WeatherData weatherData;

    public CurrentConditionsDisplay(WeatherData weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }

    public void update(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        display();
    }

    public void display() {
        System.out.println("Current conditions: " + temperature
            + "F degrees and " + humidity + "% humidity");
    }
}
```

there are no Dumb Questions

Q: Is `update()` the best place to call `display()`?

A: In this simple example it made sense to call `display()` when the values changed. However, you're right; there are much better ways to design the way the data gets displayed. We'll see this when we get to the Model-View-Controller pattern.

Q: Why did you store a reference to the `WeatherData` Subject? It doesn't look like you use it again after the constructor.

A: True, but in the future we may want to un-register ourselves as an observer and it would be handy to already have a reference to the subject.



Power up the Weather Station

1 First, let's create a test harness.

The Weather Station is ready to go. All we need is some code to glue everything together. We'll be adding some more displays and generalizing things in a bit. For now, here's our first attempt:

```
public class WeatherStation {

    public static void main(String[] args) {
        WeatherData weatherData = new WeatherData();

        CurrentConditionsDisplay currentDisplay =
            new CurrentConditionsDisplay(weatherData);
        StatisticsDisplay statisticsDisplay = new StatisticsDisplay(weatherData);
        ForecastDisplay forecastDisplay = new ForecastDisplay(weatherData);

        weatherData.setMeasurements(80, 65, 30.4f);
        weatherData.setMeasurements(82, 70, 29.2f);
        weatherData.setMeasurements(78, 90, 29.2f);
    }
}
```

First, create the WeatherData object.

If you don't want to download the code, you can comment out these two lines and run it.

Create the three displays and pass them the WeatherData object.

Simulate new weather measurements.

2 Run the code and let the Observer Pattern do its magic.

```
File Edit Window Help StormyWeather
%java WeatherStation
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!
Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
%
```



Sharpen your pencil

Johnny Hurricane, Weather-O-Rama's CEO, just called and they can't possibly ship without a Heat Index display element. Here are the details.

The heat index is an index that combines temperature and humidity to determine the apparent temperature (how hot it actually feels). To compute the heat index, you take the temperature, T , and the relative humidity, RH , and use this formula:

heatindex =

$$16.923 + 1.85212 * 10^{-1} * T + 5.37941 * RH - 1.00254 * 10^{-1} * T * RH + 9.41695 * 10^{-3} * T^2 + 7.28898 * 10^{-3} * RH^2 + 3.45372 * 10^{-4} * T^2 * RH - 8.14971 * 10^{-4} * T * RH^2 + 1.02102 * 10^{-5} * T^2 * RH^2 - 3.8646 * 10^{-5} * T^3 + 2.91583 * 10^{-5} * RH^3 + 1.42721 * 10^{-6} * T^3 * RH + 1.97483 * 10^{-7} * T * RH^3 - 2.18429 * 10^{-8} * T^3 * RH^2 + 8.43296 * 10^{-10} * T^2 * RH^3 - 4.81975 * 10^{-11} * T^3 * RH^3$$

So get typing!

Just kidding. Don't worry, you won't have to type that formula in; just create your own *HeatIndexDisplay.java* file and copy the formula from *heatindex.txt* into it.

You can get *heatindex.txt* from wickedlysmart.com.

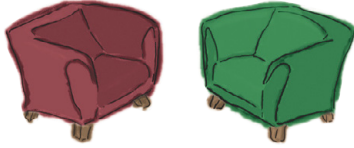
How does it work? You'd have to refer to *Head First Meteorology*, or try asking someone at the National Weather Service (or try a web search).

When you finish, your output should look like this:

Here's what changed in this output.

```
File Edit Window Help OverDaRainbow
%java WeatherStation
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!
Heat index is 82.95535
Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Heat index is 86.90124
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
Heat index is 83.64967
%
```

Fireside Chats



Tonight's talk: **Subject and Observer spar over the right way to get state information to the Observer.**

Subject:

I'm glad we're finally getting a chance to chat in person.

Well, I do my job, don't I? I always tell you what's going on.... Just because I don't really know who you are doesn't mean I don't care. And besides, I do know the most important thing about you—you implement the Observer interface.

Oh yeah, like what?

Well, excuuuse me. I have to send my state with my notifications so all you lazy Observers will know what happened!

Well...I guess that might work. I'd have to open myself up even more, though, to let all you Observers come in and get the state that you need. That might be kind of dangerous. I can't let you come in and just snoop around looking at everything I've got.

Observer:

Really? I thought you didn't care much about us Observers.

Yeah, but that's just a small part of who I am. Anyway, I know a lot more about you...

Well, you're always passing your state around to us Observers so we can see what's going on inside you. Which gets a little annoying at times...

Okay, wait just a minute here; first, we're not lazy, we just have other stuff to do in between your oh-so-important notifications, Mr. Subject, and second, why don't you let us come to you for the state we want rather than pushing it out to just everyone?

Subject:

Yes, I could let you **pull** my state. But won't that be less convenient for you? If you have to come to me every time you want something, you might have to make multiple method calls to get all the state you want. That's why I like **push** better...then you have everything you need in one notification.

Well, as I like to say, don't call us, we'll call you!
But I'll give it some thought.

You never know, hell *could* freeze over.

Indeed.

Observer:

Why don't you just write some public getter methods that will let us pull out the state we need?

Don't be so pushy! There are so many different kinds of us Observers, there's no way you can anticipate everything we need. Just let us come to you to get the state we need. That way, if some of us only need a little bit of state, we aren't forced to get it all. It also makes things easier to modify later. Say, for example, you expand yourself and add some more state. If you use pull, you don't have to go around and change the update calls on every observer; you just need to change yourself to allow more getter methods to access our additional state.

I won't hold my breath.

I see, always the wise guy...

Looking for the Observer Pattern in the Wild

The Observer Pattern is one of the most common patterns in use, and you'll find plenty of examples of the pattern being used in many libraries and frameworks. If we look at the Java Development Kit (JDK), for instance, both the JavaBeans and Swing libraries make use of the Observer Pattern. The pattern's not limited to Java either; it's used in JavaScript's events and in Cocoa and Swift's Key-Value Observing protocol, to name a couple of other examples. One of the advantages of knowing design patterns is recognizing and quickly understanding the design motivation in your favorite libraries. Let's take a quick diversion into the Swing library to see how Observer is used.



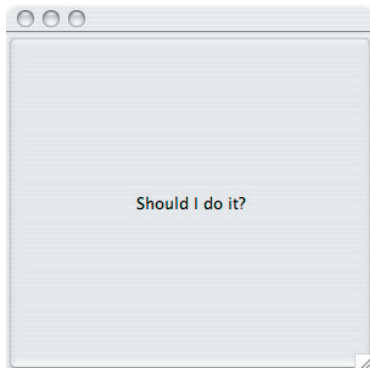
If you're curious about the Observer Pattern in JavaBeans, check out the `PropertyChangeListener` interface.

The Swing library

You probably already know that Swing is Java's GUI toolkit for user interfaces. One of the most basic components of that toolkit is the `JButton` class. If you look up `JButton`'s superclass, `AbstractButton`, you'll find that it has a lot of add/remove listener methods. These methods allow you to add and remove observers—or, as they are called in Swing, listeners—to listen for various types of events that occur on the Swing component. For instance, an `ActionListener` lets you “listen in” on any types of actions that might occur on a button, like a button press. You'll find various types of listeners all over the Swing API.

A little life-changing application

Okay, our application is pretty simple. You've got a button that says, “Should I do it?” and when you click on that button the listeners (observers) get to answer the question in any way they want. We're implementing two such listeners, called the `AngelListener` and the `DevilListener`. Here's how the application behaves:



Here's our fancy interface.



And here's the output when we click on the button.

Devil answer →
Angel answer →

```
File Edit Window Help HeMadeMeDolt
%java SwingObserverExample
Come on, do it!
Don't do it, you might regret it!
%
```

Coding the life-changing application

This life-changing application requires very little code. All we need to do is create a `JButton` object, add it to a `JFrame`, and set up our listeners. We're going to use inner classes for the listeners, which is a common technique in Swing programming. If you aren't up on inner classes or Swing, you might want to review the Swing chapter in your favorite Java reference guide.

```
public class SwingObserverExample {
    JFrame frame;

    public static void main(String[] args) {
        SwingObserverExample example = new SwingObserverExample();
        example.go();
    }

    public void go() {
        frame = new JFrame();

        JButton button = new JButton("Should I do it?");
        button.addActionListener(new AngelListener());
        button.addActionListener(new DevilListener());

        // Set frame properties here

    }

    class AngelListener implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            System.out.println("Don't do it, you might regret it!");
        }
    }

    class DevilListener implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            System.out.println("Come on, do it!");
        }
    }
}
```

Simple Swing application that just creates a frame and throws a button in it.

Makes the devil and angel objects listeners (observers) of the button.

Code to set up the frame goes here.

Here are the class definitions for the observers, defined as inner classes (but they don't have to be).

Rather than `update()`, the `actionPerformed()` method gets called when the state in the subject (in this case the button) changes.



Serious Coding

Lambda expressions were added in Java 8. If you aren't familiar with them, don't worry about it; you can continue using inner classes for your Swing observers.

How about taking your use of the Observer Pattern even further? By using a lambda expression rather than an inner class, you can skip the step of creating an ActionListener object. With a lambda expression, we create a function object instead, and *the function object is the observer*. And, when you pass that function object to `addActionListener()`, Java ensures its signature matches `actionPerformed()`, the one method in the ActionListener interface.

Later, when the button is clicked, the button object notifies its observers—including the function objects created by the lambda expressions—that it's been clicked, and calls each listener's `actionPerformed()` method.

Let's take a look at how you'd use lambda expressions as observers to simplify our previous code:

The updated code, using lambda expressions:

```
public class SwingObserverExample {
    JFrame frame;
    public static void main(String[] args) {
        SwingObserverExample example = new SwingObserverExample();
        example.go();
    }
    public void go() {
        frame = new JFrame();

        JButton button = new JButton("Should I do it?");
        button.addActionListener(event ->
            System.out.println("Don't do it, you might regret it!"));
        button.addActionListener(event ->
            System.out.println("Come on, do it!"));

        // Set frame properties here
    }
}
```

We've replaced the `AngellListener` and `DevilListener` objects with lambda expressions that implement the same functionality that we had before.

When you click the button, the function objects created by the lambda expressions are notified and the method they implement is run.

Using lambda expressions makes this code a lot more concise.

We've removed the two ActionListener classes (`DevilListener` and `AngellListener`) completely.

For more on lambda expressions, check out the Java docs.

there are no Dumb Questions

Q: I thought Java had **Observer** and **Observable** classes?

A: Good catch. Java used to provide an **Observable** class (the **Subject**) and an **Observer** interface, which you could use to help integrate the **Observer Pattern** in your code. The **Observable** class provided methods to add, delete, and notify observers, so that you didn't have to write that code. And the **Observer** interface provided an interface just like ours, with one `update()` method. These classes were deprecated in Java 9. Folks find it easier to support the basic **Observer Pattern** in their own code, or want something more robust, so the **Observer/Observable** classes are being phased out.

Q: Does Java offer other built-in support for **Observer** to replace those classes?

A: JavaBeans offers built-in support through **PropertyChangeEvent**s that are generated when a **Bean** changes a particular kind of property, and sends notifications to **PropertyChangeListener**s. There are also related publisher/subscriber components in the **Flow API** for handling asynchronous streams.

Q: Should I expect notifications from a **Subject** to its **Observers** to arrive in a specific order?

A: With Java's implementations of **Observer**, the **JDK** developers specifically advise you to **not** depend on any specific notification order.

I was thinking about the push/pull discussion we had earlier. Would it generalize the code a bit more if we allowed the displays to pull their data from the **WeatherData** object as needed? That might make it easier to add new displays in the future.

That's a good idea.

In our current **Weather Station** design, we are *pushing* all three pieces of data to the `update()` method in the displays, even if the displays don't need all these values. That's okay, but what if **Weather-O-Rama** adds another data value later, like wind speed? Then we'll have to change all the `update()` methods in all the displays, even if most of them don't need or want the wind speed data.

Now, whether we pull or push the data to the **Observer** is an implementation detail, but in a lot of cases it makes sense to let **Observers** retrieve the data they need rather than passing more and more data to them through the `update()` method. After all, over time, this is an area that may change and grow unwieldy. And, we know **CEO Johnny Hurricane** is going to want to expand the **Weather Station** and sell more displays, so let's take another pass at the design and see if we can make it even easier to expand in the future.

Updating the **Weather Station** code to allow **Observers** to *pull* the data they need is a pretty straightforward exercise. All we need to do is make sure the **Subject** has getter methods for its data, and then change our **Observers** to use them to pull the data that's appropriate for their needs. Let's do that.





Meanwhile, back at Weather-O-Rama

There's another way of handling the data in the Subject: we can rely on the Observers to pull it from the Subject as needed. Right now, when the Subject's data changes, we *push* the new values for temperature, humidity, and pressure to the Observers, by passing that data in the call to `update()`.

Let's set things up so that when an Observer is notified of a change, it calls getter methods on the Subject to *pull* the values it needs.

To switch to using pull, we need to make a few small changes to our existing code.

For the Subject to send notifications...

- 1 We'll modify the `notifyObservers()` method in `WeatherData` to call the method `update()` in the Observers with no arguments:

```
public void notifyObservers() {
    for (Observer observer : observers) {
        observer.update();
    }
}
```

For an Observer to receive notifications...

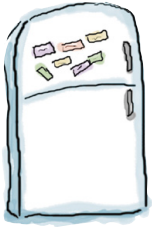
- 1 Then we'll modify the `Observer` interface, changing the signature of the `update()` method so that it has no parameters:

```
public interface Observer {
    public void update();
}
```

- 2 And finally, we modify each concrete Observer to change the signature of its respective `update()` methods and get the weather data from the Subject using the `WeatherData`'s getter methods. Here's the new code for the `CurrentConditionsDisplay` class:

```
public void update() {
    this.temperature = weatherData.getTemperature();
    this.humidity = weatherData.getHumidity();
    display();
}
```

Here we're using the Subject's getter methods that were supplied with the code in `WeatherData` from Weather-O-Rama.



Code Magnets

The ForecastDisplay class is all scrambled up on the fridge. Can you reconstruct the code snippets to make it work? Some of the curly braces fell on the floor and they were too small to pick up, so feel free to add as many of those as you need!

```
public ForecastDisplay(WeatherData
weatherData) {
```

```
display();
```

```
weatherData.registerObserver(this);
```

```
public class ForecastDisplay implements
Observer, DisplayElement {
```

```
public void display() {
    // display code here
}
```

```
lastPressure = currentPressure;
currentPressure = weatherData.getPressure();
```

```
private float currentPressure = 29.92f;
private float lastPressure;
```

```
this.weatherData = weatherData;
```

```
public void update() {
```

```
}
```

```
private WeatherData weatherData;
```

Test Drive the new code



Okay, you've got one more display to update, the Avg/Min/Max display. Go ahead and do that now!

Just to be sure, let's run the new code...

Here's what we got.

Look! This just arrived!

```
File Edit Window Help TryThisAtHome

%java WeatherStation
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!
Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
%
```



Weather-O-Rama, Inc.
100 Main Street
Tornado Alley, OK 45021

Wow!

Your design is fantastic. Not only did you quickly create all three displays that we asked for, you've created a general design that allows anyone to create new display, and even allows users to add and remove displays at runtime!

Ingenious!

Until our next engagement,

Johnny Hurricane



Tools for your Design Toolbox

Welcome to the end of Chapter 2. You've added a few new things to your OO toolbox...

OO Basics

Abstraction

rtion

hism

ce

OO Principles

Encapsulate what varies.

Favor composition over inheritance.

Program to interfaces, not implementations.

Strive for loosely coupled designs between objects that interact.

Here's your newest principle. Remember, loosely coupled designs are much more flexible and resilient to change.

OO Patterns

Strat
encap
inter
vary

Observer - defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically

A new pattern for communicating state to a set of objects in a loosely coupled manner. We haven't seen the last of the Observer Pattern—just wait until we talk about MVC!



BULLET POINTS

- The Observer Pattern defines a one-to-many relationship between objects.
- Subjects update Observers using a common interface.
- Observers of any concrete type can participate in the pattern as long as they implement the Observer interface.
- Observers are loosely coupled in that the Subject knows nothing about them, other than that they implement the Observer interface.
- You can push or pull data from the Subject when using the pattern (pull is considered more “correct”).
- Swing makes heavy use of the Observer Pattern, as do many GUI frameworks.
- You'll also find the pattern in many other places, including RxJava, JavaBeans, and RMI, as well as in other language frameworks, like Cocoa, Swift, and JavaScript events.
- The Observer Pattern is related to the Publish/Subscribe Pattern, which is for more complex situations with multiple Subjects and/or multiple message types.
- The Observer Pattern is a commonly used pattern, and we'll see it again when we learn about Model-View-Controller.



Design Principle Challenge

For each design principle, describe how the Observer Pattern makes use of the principle.

Design Principle

Identify the aspects of your application that vary and separate them from what stays the same.

Design Principle

Program to an interface, not an implementation.

Design Principle

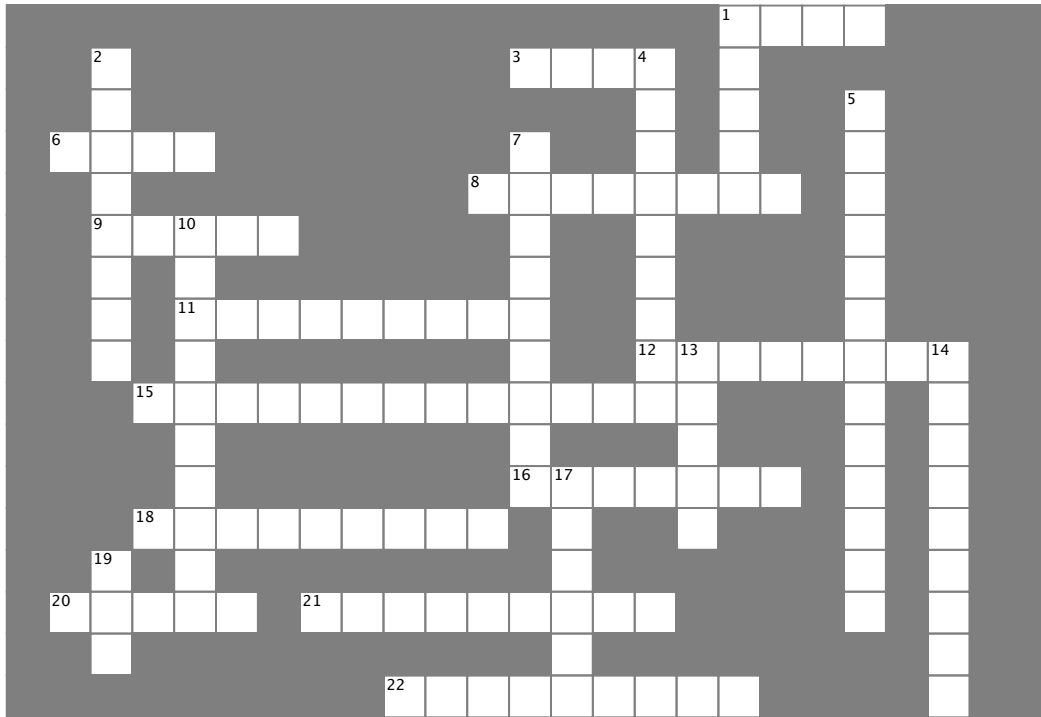
Favor composition over inheritance.

This is a hard one. Hint: think about how observers and subjects work together.



Design Patterns Crossword

Time to give your right brain something to do again!
All of the solution words are from Chapters 1 & 2.



ACROSS

1. One Subject likes to talk to _____ observers.
3. Subject initially wanted to _____ all the data to Observer.
6. CEO almost forgot the _____ index display.
8. CurrentConditionsDisplay implements this interface.
9. Java framework with lots of Observers.
11. A Subject is similar to a _____.
12. Observers like to be _____ when something new happens.
15. How to get yourself off the Observer list.
16. Lori was both an Observer and a _____.
18. Subject is an _____.
20. You want to keep your coupling _____.
21. Program to an _____ not an implementation.
22. Devil and Angel are _____ to the button.

DOWN

1. He didn't want any more ints, so he removed himself.
2. Temperature, humidity, and _____.
4. Weather-O-Rama's CEO is named after this kind of storm.
5. He says you should go for it.
7. The Subject doesn't have to know much about the _____.
10. The WeatherData class _____ the Subject interface.
13. Don't count on this for notification.
14. Observers are _____ on the Subject.
17. Implement this method to get notified.
19. Jill got one of her own.



Sharpen your pencil Solution

Based on our first implementation, which of the following apply? (Choose all that apply.)

- ☒ A. We are coding to concrete implementations, not interfaces.
- ☒ B. For every new display element, we need to alter code.
- ☒ C. We have no way to add display elements at runtime.
- ☐ D. The display elements don't implement a common interface.
- ☒ E. We haven't encapsulated what changes.
- ☐ F. We are violating encapsulation of the WeatherData class.



Design Principle Challenge Solution

Design Principle

Identify the aspects of your application that vary and separate them from what stays the same.

The thing that varies in the Observer Pattern is the state of the Subject and the number and types of Observers. With this pattern, you can vary the objects that are dependent on the state of the Subject, without having to change that Subject. That's called planning ahead!

Design Principle

Program to an interface, not an implementation.

Both the Subject and Observers use interfaces. The Subject keeps track of objects implementing the Observer interface, while the Observers register with, and get notified by, the Subject interface. As we've seen, this keeps things nice and loosely coupled.

Design Principle

Favor composition over inheritance.

The Observer Pattern uses composition to compose any number of Observers with their Subject. These relationships aren't set up by some kind of inheritance hierarchy. No, they are set up at runtime by composition!



Code Magnets Solution

The ForecastDisplay class is all scrambled up on the fridge. Can you reconstruct the code snippets to make it work? Some of the curly braces fell on the floor and they were too small to pick up, so feel free to add as many of those as you need! Here's our solution.

```
public class ForecastDisplay implements
Observer, DisplayElement {
```

```
    private float currentPressure = 29.92f;
    private float lastPressure;
```

```
    private WeatherData weatherData;
```

```
    public ForecastDisplay(WeatherData
weatherData) {
```

```
        this.weatherData = weatherData;
```

```
        weatherData.registerObserver(this);
```

```
    }
```

```
    public void update() {
```

```
        lastPressure = currentPressure;
        currentPressure = weatherData.getPressure();
```

```
        display();
```

```
    }
```

```
    public void display() {
        // display code here
    }
```

```
}
```

[illegible]

3 the Decorator Pattern

✧ **Decorating Objects** ✧

I used to think real men subclassed everything. That was until I learned the power of extension at runtime, rather than at compile time. Now look at me!



Just call this chapter “Design Eye for the Inheritance Guy.”

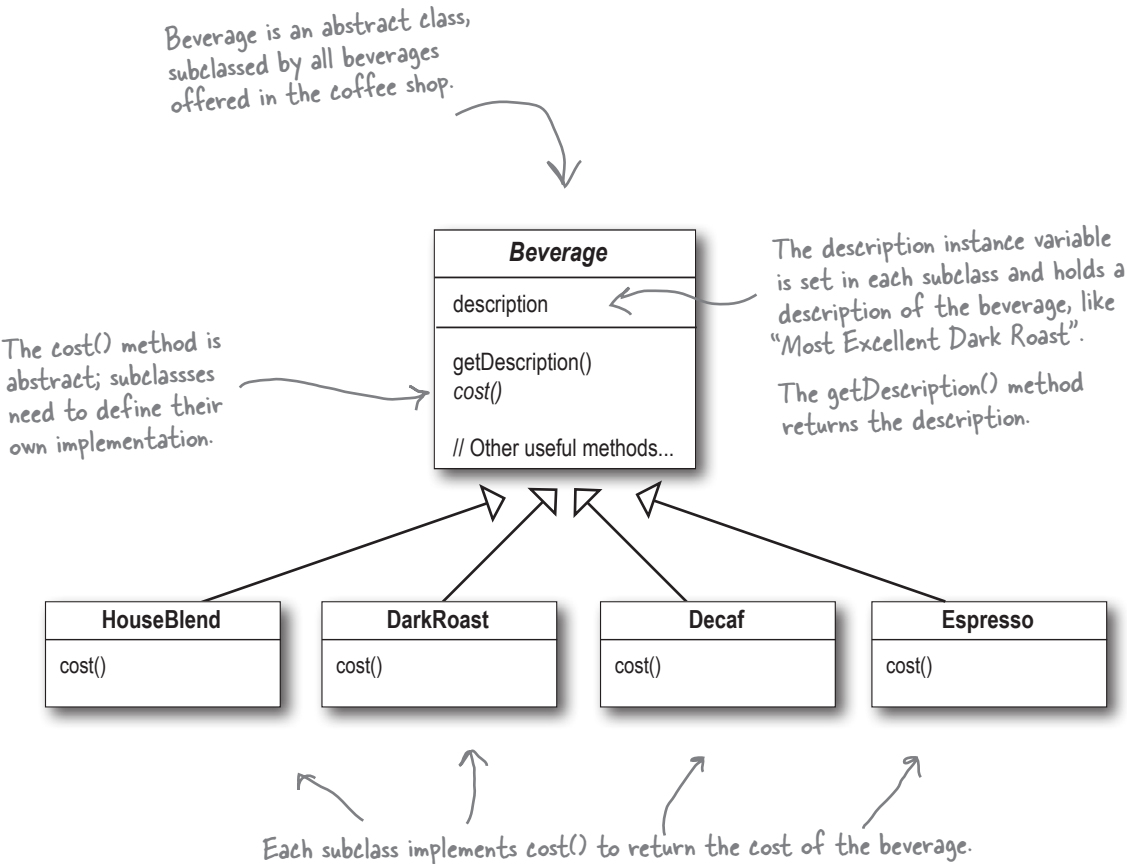
We’ll re-examine the typical overuse of inheritance and you’ll learn how to decorate your classes at runtime using a form of object composition. Why? Once you know the techniques of decorating, you’ll be able to give your (or someone else’s) objects new responsibilities *without making any code changes to the underlying classes*.

Welcome to Starbuzz Coffee

Starbuzz Coffee has made a name for itself as the fastest-growing coffee shop around. If you've seen one on your local corner, look across the street; you'll see another one.

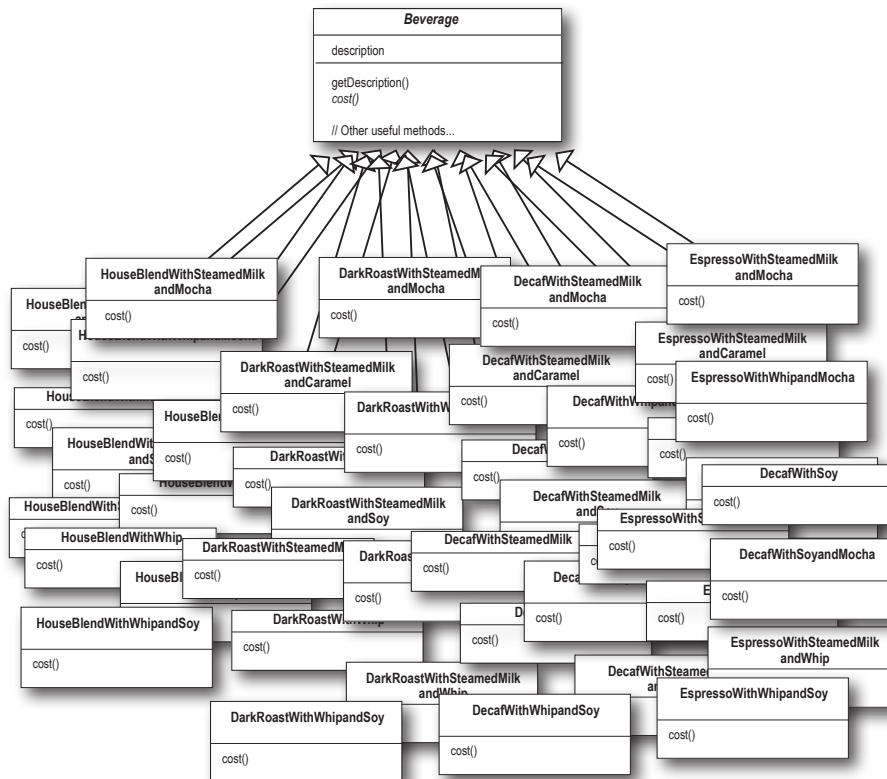
Because they've grown so quickly, they're scrambling to update their ordering systems to match their beverage offerings.

When they first went into business they designed their classes like this...



In addition to your coffee, you can also ask for several condiments like steamed milk, soy, and mocha (otherwise known as chocolate), and have it all topped off with whipped milk. Starbuzz charges a bit for each condiment, so they really need to get them built into their order system.

Here's their first attempt...



Whoa!
Can you say
"class explosion"?

Each cost method computes the cost of the coffee along with the other condiments in the order.



BRAIN POWER

It's pretty obvious that Starbuzz has created a maintenance nightmare for themselves. What happens when the price of milk goes up? What do they do when they add a new caramel topping?

Thinking beyond the maintenance problem, which of the design principles that we've covered so far are they violating?

Hint: they're violating two of them in a big way!

This is stupid; why do we need all these classes? Can't we just use instance variables and inheritance in the superclass to keep track of the condiments?



Well, let's give it a try. Let's start with the Beverage base class and add instance variables to represent whether or not each beverage has milk, soy, mocha, and whip...

Beverage	
description	
milk	
soy	
mocha	
whip	
getDescription()	
cost()	
hasMilk()	
setMilk()	
hasSoy()	
setSoy()	
hasMocha()	
setMocha()	
hasWhip()	
setWhip()	
// Other useful methods..	

New boolean values for each condiment.

Now we'll implement `cost()` in `Beverage` (instead of keeping it abstract), so that it can calculate the costs associated with the condiments for a particular beverage instance. Subclasses will still override `cost()`, but they will also invoke the super version so that they can calculate the total cost of the basic beverage plus the costs of the added condiments.

These get and set the boolean values for the condiments.